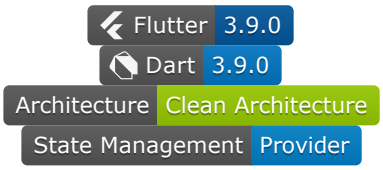
🚀 EventUp - Developer Documentation



EventUp Developer Documentation

Architecture • Setup • Contributing • API

📋 İçindekiler

🎯 Proje Genel Bakış

EventUp, kullanıcıların ilgi alanlarına göre etkinlik keşfedip katılabileceği, diğer katılımcılarla anlık mesajlaşabileceği modern bir Flutter mobil uygulamasıdır.

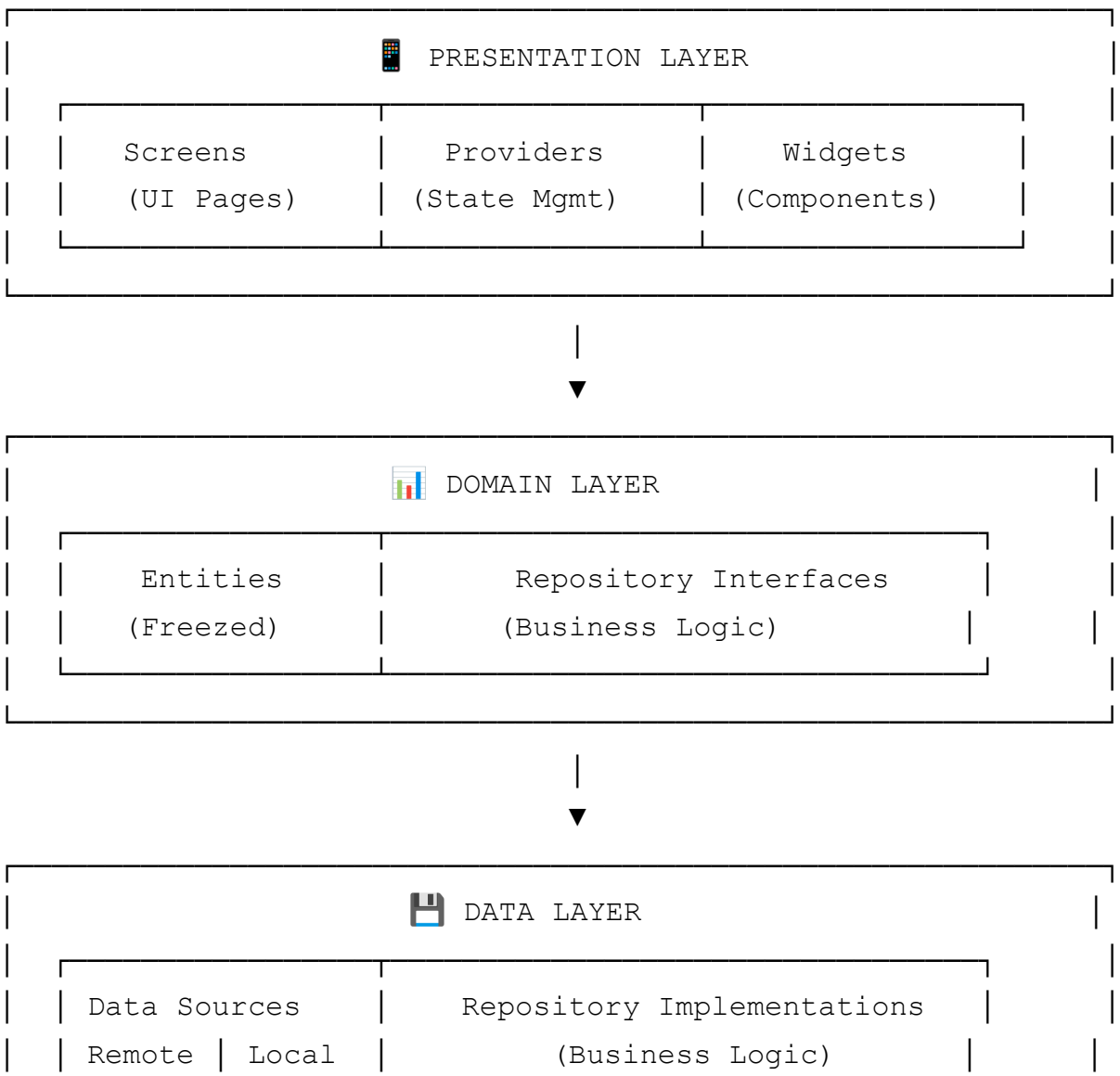
Ana Özellikler

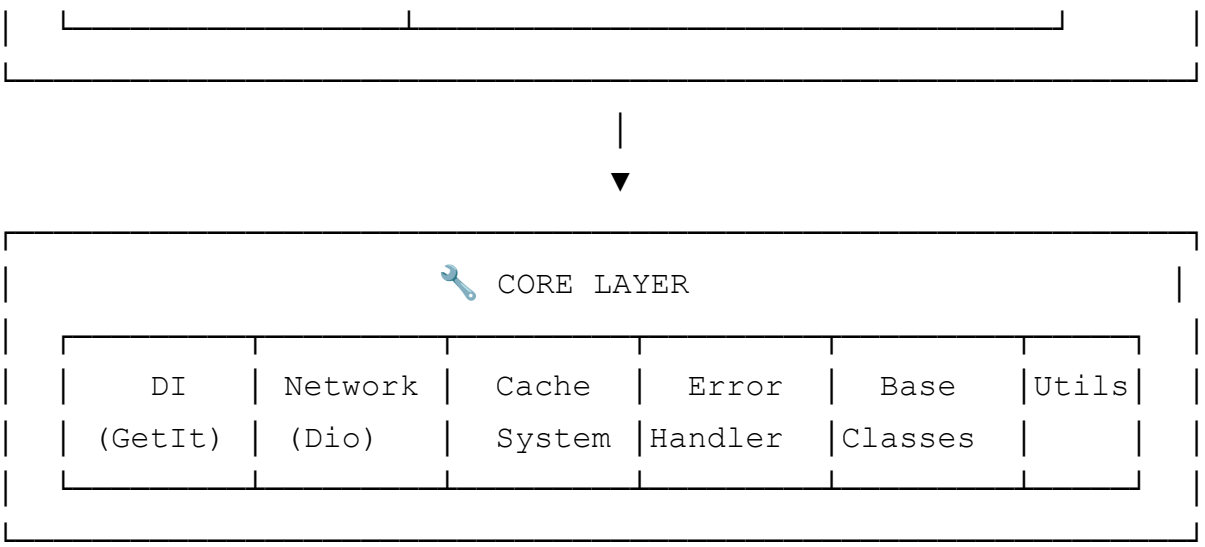
- ✅ Güvenli kullanıcı kimlik doğrulama
- ✅ Etkinlik keşfetme ve katılım
- ✅ Real-time mesajlaşma (SignalR)
- ✅ Profil yönetimi ve ilgi alanları
- ✅ Modern UI/UX tasarım

🏗️ Mimari Genel Bakış

Proje **Clean Architecture** prensiplerine uygun şekilde tasarlanmıştır ve **SOLID** prensiplerini takip eder.

Mimari Katmanları





Detaylı Katman Açıklamaları

📱 Presentation Layer (UI & State Management)

- **Providers**: State management için Provider pattern
- **Screens**: Kullanıcı arayüzü ekranları
- **Widgets**: Yeniden kullanılabilir UI bileşenleri

```
// Provider Örneği
@injectable
class EventsProvider extends BaseProvider with ListProviderMixin<Event> {
  Future<void> loadEvents() async {
    await execute(
      () => _repository.getEvents(),
      onSuccess: setItems,
    );
  }
}
```

📊 Domain Layer (Business Logic)

- **Entities**: Immutable business objects (Freezed ile)
- **Repository Interfaces**: Business logic abstraction

```
// Entity Örneği
@freezed
class Event with _$Event {
  const factory Event({
    required String id,
    required String title,
```

```
    required DateTime date,
  }) = _Event;
}
```

💾 Data Layer (Data Access)

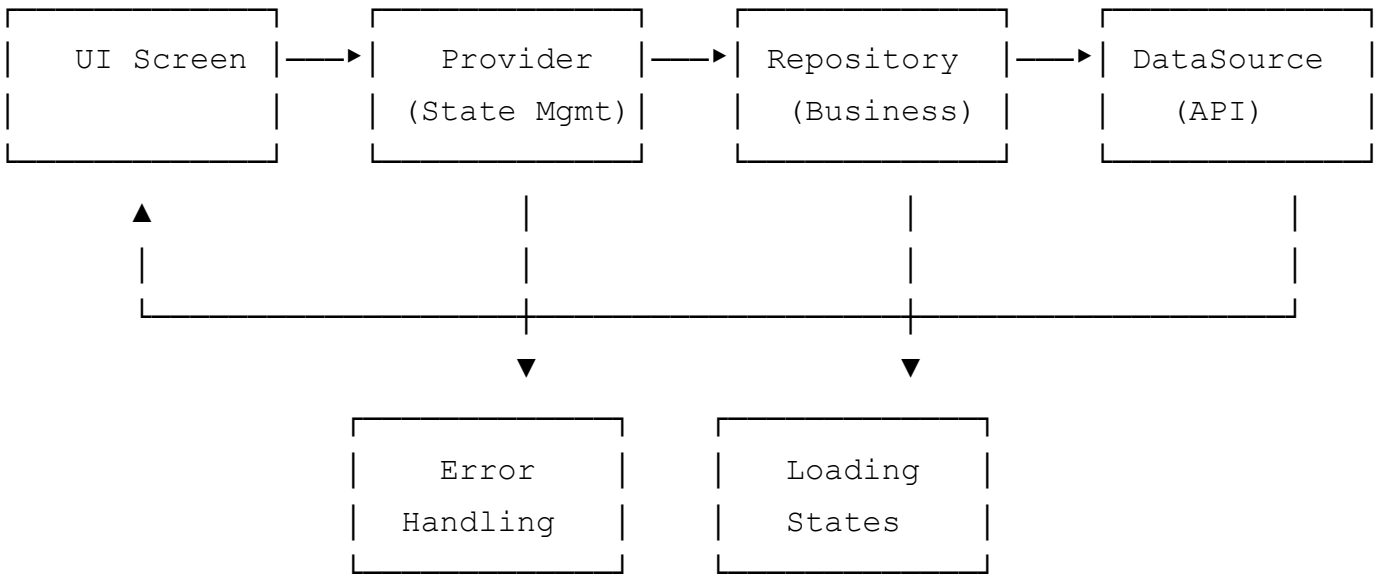
- **Data Sources**: API calls ve local storage
- **Repository Implementations**: Data operations

```
// Repository Implementation
@LazySingleton()
class EventsRepository {
  Future<Result<List<Event>>> getEvents() async {
    // Business logic implementation
  }
}
```

🔧 Core Layer (Infrastructure)

- **DI**: Dependency injection (GetIt + Injectable)
- **Network**: HTTP client (Dio + Interceptors)
- **Cache**: Memory caching system
- **Error**: Exception & failure handling
- **Base**: Base classes ve mixins
- **Utils**: Utilities ve helpers

Veri Akışı Diyagramı



Hata Yönetimi Akışı

```
HTTP Error/Exception
         |
         ▼
Dio Interceptor (Transform)
         |
         ▼
Repository (catch & transform)
         |
         ▼
Result<T> (Success/Failed)
         |
         ▼
Provider (handle status)
         |
         ▼
UI (display error/success)
```

Dependency Injection Yapısı

```
main.dart
    |
    ▼
configureDependencies()
    |
    ▼
GetIt Container
    ├── DataSources (LazySingleton)
    ├── Repositories (LazySingleton)
    └── Providers (Factory)
    |
    ▼
Provider Registration in MultiProvider
```

🛠️ Geliştirme Kurulumu

Gereksinimler

- **Flutter**: ^3.9.0

- **Dart**: ^3.9.0
- **VS Code** / **Android Studio**
- **Git**

Kurulum Adımları

1. **Repository'yi klonlayın**

   ```
   git clone <repository-url>
   cd event_up_app
   ```

2. **Dependencies'leri yükleyin**

   ```
   flutter pub get
   ```

3. **Code generation'ı çalıştırın**

   ```
   flutter packages pub run build_runner build
   ```

4. **Uygulamayı çalıştırın**

   ```
   flutter run
   ```

Environment Configuration

Uygulama environment-specific ayarları `lib/config/environment.dart` dosyasında yönetir:

```
// Development
flutter run --dart-define=API_URL=http://localhost:5001/api

// Production
flutter run --dart-define=API_URL=https://api.eventup.com
```

🔌 Dependency Injection

Proje, **GetIt** ve **Injectable** kullanarak güçlü bir DI sistemi implement eder.

Setup

```
// lib/core/di/injection.dart
final GetIt getIt = GetIt.instance;

@InjectableInit()
Future<void> configureDependencies() async {
  await getIt.init();
}
```

Dependency Registration

```
// DataSource Registration
@LazySingleton()
class AuthRemoteDataSource {
  // Implementation
}

// Repository Registration
@LazySingleton()
class AuthRepository {
  // Implementation
}

// Provider Registration
@injectable
class AuthProvider extends BaseProvider {
  // Implementation
}
```

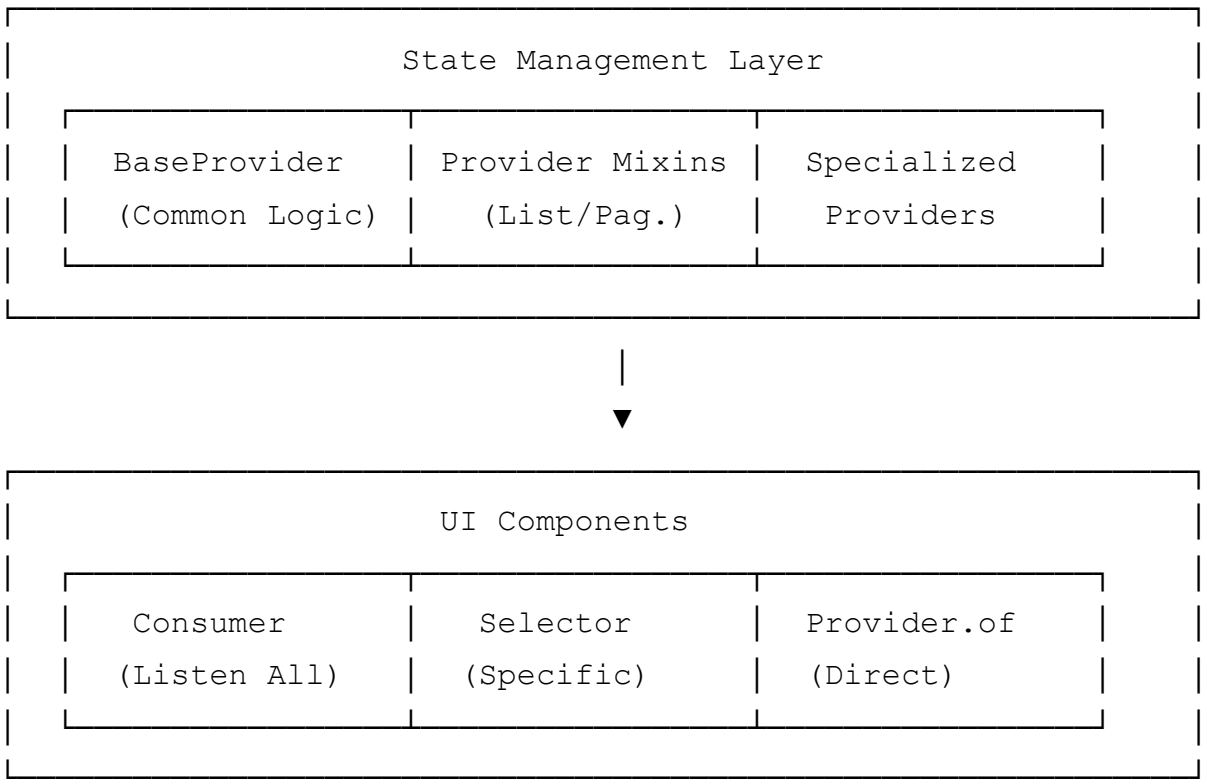
Usage in Code

```
// Provider injection in main.dart
ChangeNotifierProvider<AuthProvider>(
  create: (_) => getIt<AuthProvider>(),
)

// Direct injection
final authProvider = getIt<AuthProvider>();
```

State Management

Proje **Provider** pattern kullanır ve `BaseProvider` ile tutarlılık sağlar.

State Management Mimarisi



BaseProvider Yapısı

`BaseProvider` , tüm provider'lar için ortak state management mantığını sağlar:

```
abstract class BaseProvider extends ChangeNotifier {
  bool _isLoading = false;
  Failure? _failure;

  // State getters
  bool get isLoading => _isLoading;
  Failure? get failure => _failure;
  String? get errorMessage => _failure?.message;
  bool get hasError => _failure != null;

  // State setters
  @protected
  void setLoading(bool value) {
    _isLoading = value;
```

```
    notifyListeners();
  }

  @protected
  void setFailure(Failure? failure) {
    _failure = failure;
    notifyListeners();
  }

  // Execute operation with automatic state management
  @protected
  Future<T?> execute<T>(
    Future<Result<T>> Function() operation, {
    void Function(T data)? onSuccess,
    void Function(Failure failure)? onFailure,
    bool showLoading = true,
  }) async {
    if (showLoading) setLoading(true);
    setFailure(null);

    try {
      final result = await operation();
      return result.fold(
        onSuccess: (data) {
          onSuccess?.call(data);
          return data;
        },
        onFailure: (failure) {
          setFailure(failure);
          onFailure?.call(failure);
          return null;
        },
      );
    } catch (e) {
      final failure = UnknownFailure(message: e.toString());
      setFailure(failure);
      onFailure?.call(failure);
      return null;
    } finally {
      if (showLoading) setLoading(false);
    }
  }
}
```

Provider Mixins

ListProviderMixin

Liste yönetimi için kullanılır:

```
mixin ListProviderMixin<T> on BaseProvider {
  List<T> _items = [];

  List<T> get items => List.unmodifiable(_items);
  bool get isEmpty => _items.isEmpty;
  bool get hasItems => _items.isNotEmpty;
  int get itemCount => _items.length;

  @protected
  void setItems(List<T> items) {
    _items = items;
    notifyListeners();
  }

  @protected
  void addItem(T item) {
    _items.add(item);
    notifyListeners();
  }

  @protected
  void removeItem(T item) {
    _items.remove(item);
    notifyListeners();
  }

  @protected
  void clearItems() {
    _items = [];
    notifyListeners();
  }
}
```

PaginationProviderMixin

Sayfalama desteği için kullanılır:

```
mixin PaginationProviderMixin<T> on BaseProvider {
  int _currentPage = 1;
  int _totalPages = 1;
  bool _hasMore = true;
  List<T> _items = [];

  int get currentPage => _currentPage;
  int get totalPages => _totalPages;
  bool get hasMore => _hasMore;
  List<T> get items => List.unmodifiable(_items);

  @protected
  void setPaginatedData({
    required List<T> items,
    required int currentPage,
    required int totalPages,
    bool append = false,
  }) {
    _currentPage = currentPage;
    _totalPages = totalPages;
    _hasMore = currentPage < totalPages;

    if (append) {
      _items.addAll(items);
    } else {
      _items = items;
    }
    notifyListeners();
  }
}
```

Result Pattern

Error handling için functional approach:

```
sealed class Result<T> {
  const Result();

  bool get isSuccess => this is Success<T>;
  bool get isFailure => this is Failed<T>;

  T? get dataOrNull => isSuccess ? (this as Success<T>).value : null;
```

```
  Failure? get failureOrNull => isFailure ? (this as Failed<T>).failure :

  R fold<R>({
    required R Function(T data) onSuccess,
    required R Function(Failure failure) onFailure,
  }) {
    if (isSuccess) {
      return onSuccess((this as Success<T>).value);
    } else {
      return onFailure((this as Failed<T>).failure);
    }
  }
}

final class Success<T> extends Result<T> {
  const Success(this.value, {this.message});
  final T value;
  final String? message;
}

final class Failed<T> extends Result<T> {
  const Failed(this.failure);
  final Failure failure;
}
```

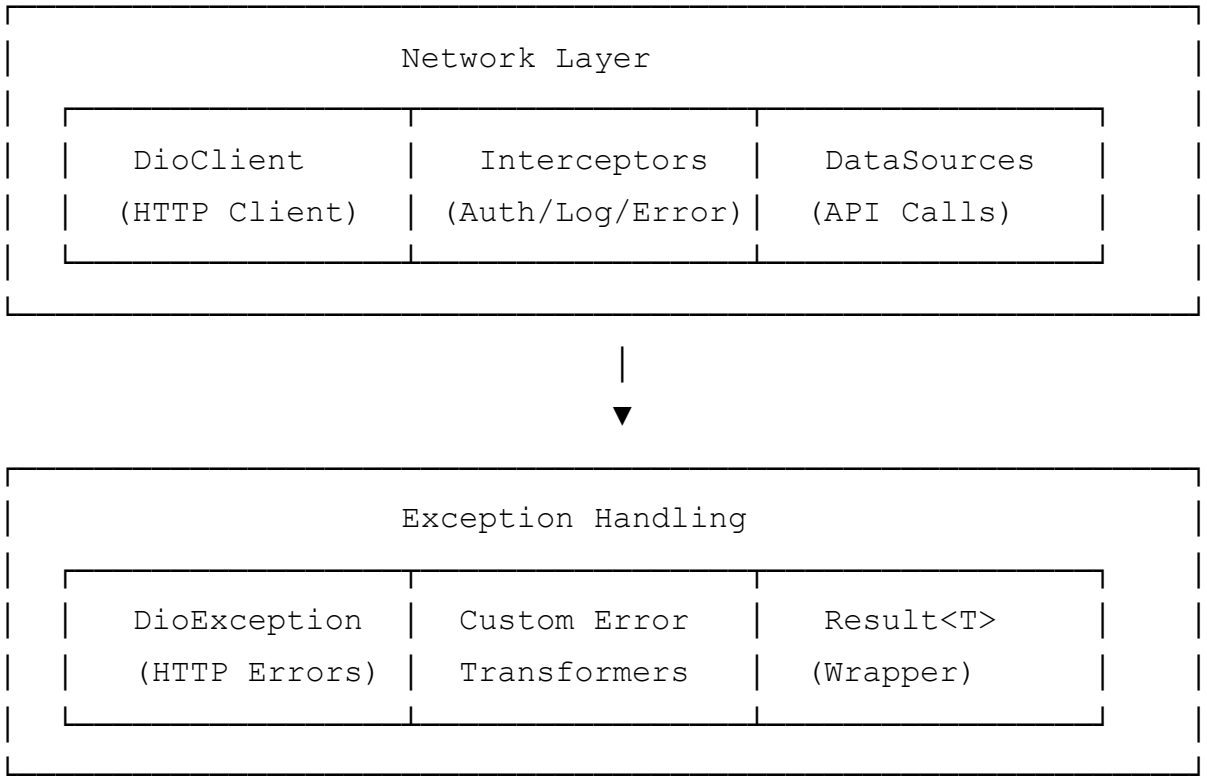
Provider Örneği

```
@injectable
class EventsProvider extends BaseProvider with ListProviderMixin<Event> {
  EventsProvider(this._eventsRepository);
  final EventsRepository _eventsRepository;

  List<Event> get events => items;

  Future<void> loadEvents({
    required String token,
    String? search,
    String? location,
  }) async {
    await execute(
      () => _eventsRepository.getFilteredEvents(
        token: token,
```

```
        search: search,
        location: location,
      ),
      onSuccess: setItems,
    );
  }
}
```

🌐 API ve Servis Katmanı

Network Layer Mimarisi



HTTP Client (DioClient)

Merkezi HTTP client yapılandırması:

```
@lazySingleton
class DioClient {
  late final Dio _dio;
  Dio get dio => _dio;

  DioClient() {
    _dio = Dio(_baseOptions);
```

```
    _setupInterceptors();
  }

  BaseOptions get _baseOptions => BaseOptions(
    baseUrl: Environment.apiUrl,
    connectTimeout: AppConstants.connectionTimeout,
    receiveTimeout: AppConstants.receiveTimeout,
    sendTimeout: AppConstants.apiTimeout,
    headers: {
      'Content-Type': 'application/json',
      'Accept': 'application/json',
    },
  );

  void _setupInterceptors() {
    _dio.interceptors.addAll([
      _LoggingInterceptor(),    // Request/Response logging
      _AuthInterceptor(),       // Token management
      _ErrorInterceptor(),      // Error transformation
    ]);
  }

  // Token management
  void setAuthToken(String? token) {
    if (token != null && token.isNotEmpty) {
      _dio.options.headers['Authorization'] = 'Bearer $token';
    } else {
      _dio.options.headers.remove('Authorization');
    }
  }
}
```

Interceptors

1. Logging Interceptor

Request ve response'ları loglar:

```
class _LoggingInterceptor extends Interceptor {
  @override
  void onRequest(RequestOptions options, RequestInterceptorHandler handle
    AppLogger.network(
      '${options.method} ${options.path}',
```

```
      method: options.method,
      url: options.uri.toString(),
    );

    if (options.data != null) {
      AppLogger.log('Request Body: ${options.data}', tag: 'DioClient');
    }
    super.onRequest(options, handler);
  }

  @override
  void onResponse(Response response, ResponseInterceptorHandler handler)
    AppLogger.success(
      'Response ${response.statusCode}: ${response.requestOptions.path}',
      tag: 'DioClient',
    );
    super.onResponse(response, handler);
  }
}
```

2. Auth Interceptor

Authentication ve token yönetimi:

```
class _AuthInterceptor extends Interceptor {
  @override
  Future<void> onError(DioException err, ErrorInterceptorHandler handler)
    // 401 (Unauthorized) durumunda token refresh
    if (err.response?.statusCode == 401) {
      AppLogger.warning('401 Unauthorized - Attempting token refresh');

      // Token refresh logic burada implementasyon yapılabilir
      // 1. Refresh token'ı local storage'dan al
      // 2. Refresh endpoint'ini çağır
      // 3. Yeni token'ı güncelle
      // 4. Original request'i retry et
    }
    super.onError(err, handler);
  }
}
```

3. Error Interceptor

HTTP hatalarını custom exception'lara çevirir:

```
class _ErrorInterceptor extends Interceptor {
  @override
  void onError(DioException err, ErrorInterceptorHandler handler) {
    switch (err.type) {
      case DioExceptionType.connectionTimeout:
      case DioExceptionType.receiveTimeout:
        throw TimeoutException(message: 'Request timeout');

      case DioExceptionType.connectionError:
        throw NetworkException(message: 'Network connection failed');

      case DioExceptionType.badResponse:
        _handleResponseError(err);
        break;

      default:
        throw UnknownException(message: 'Unknown error occurred');
    }
    super.onError(err, handler);
  }

  void _handleResponseError(DioException err) {
    final statusCode = err.response?.statusCode;
    final data = err.response?.data;

    String errorMessage = 'Server error occurred';
    if (data != null && data is Map<String, dynamic>) {
      errorMessage = data['message'] ?? errorMessage;
    }

    switch (statusCode) {
      case 400:
        throw ValidationException(message: errorMessage, statusCode: stat
      case 401:
        throw AuthenticationException(message: errorMessage, statusCode:
      case 403:
        throw AuthorizationException(message: errorMessage, statusCode: s
      case 404:
        throw NotFoundException(message: errorMessage, statusCode: status
      case 500:
        throw ServerException(message: errorMessage, statusCode: statusCo
      default:
```

```
        throw UnknownException(message: errorMessage, statusCode: statusC
      }
    }
  }
```

Repository Pattern

```
// Repository Interface (Domain Layer)
abstract class AuthRepository {
  Future<Result<UserData>> login({
    required String userNameOrEmail,
    required String password,
  });
}

// Repository Implementation (Data Layer)
@LazySingleton()
class AuthRepositoryImpl implements AuthRepository {
  AuthRepositoryImpl(
    this._remoteDataSource,
    this._localDataSource,
  );

  final AuthRemoteDataSource _remoteDataSource;
  final AuthLocalDataSource _localDataSource;

  @override
  Future<Result<UserData>> login({
    required String userNameOrEmail,
    required String password,
  }) async {
    try {
      final result = await _remoteDataSource.login(
        userNameOrEmail: userNameOrEmail,
        password: password,
      );

      if (result.isSuccess) {
        // Cache user data locally
        await _localDataSource.cacheUserData(result.data);
      }
```

```
      return result;
    } catch (e) {
      return Result.failure(e.toString());
    }
  }
}
```

Data Sources

```
// Remote Data Source
@LazySingleton()
class AuthRemoteDataSource {
  AuthRemoteDataSource(this._dioClient);
  final DioClient _dioClient;

  Future<Result<UserData>> login({
    required String userNameOrEmail,
    required String password,
  }) async {
    try {
      final response = await _dioClient.post(
        '/Auth/login',
        data: {
          'userNameOrEmail': userNameOrEmail,
          'password': password,
        },
      );

      return Result.success(UserData.fromJson(response.data));
    } catch (e) {
      return Result.failure(e.toString());
    }
  }
}

// Local Data Source
@LazySingleton()
class AuthLocalDataSource {
  AuthLocalDataSource(this._sharedPreferences);
  final SharedPreferences _sharedPreferences;

  Future<Result<String?>> getCachedAuthToken() async {
```

```
      final token = _sharedPreferences.getString('auth_token');
      return Result.success(token);
    }
  }
```

📁 Kod Yapısı

Klasör Organizasyonu

```
lib/
├── config/                  # App configuration
│   ├── app_routes.dart     # Route definitions
│   ├── app_theme.dart      # Theme configuration
│   └── environment.dart    # Environment settings
├── core/                    # Core infrastructure
│   ├── base/               # Base classes and mixins
│   ├── cache/              # Caching system
│   ├── constants/          # App constants
│   ├── di/                 # Dependency injection
│   ├── error/              # Error handling
│   ├── network/            # Network layer
│   └── utils/              # Core utilities
├── data/                    # Data layer
│   ├── datasources/        # API and local data sources
│   └── repositories/       # Repository implementations
├── domain/                  # Domain layer
│   └── entities/           # Business entities
├── presentation/            # Presentation layer
│   ├── providers/          # State management providers
│   ├── screens/            # UI screens
│   └── widgets/            # Reusable widgets
├── models/                  # Data models (legacy)
├── services/                # Legacy services (deprecated)
└── main.dart                # App entry point
```

Naming Conventions

- **Classes**: PascalCase (`AuthProvider` , `EventRepository`)
- **Files**: snake_case (`auth_provider.dart` , `event_repository.dart`)

- **Variables**: camelCase (`userName` , `isLoading`)
- **Constants**: UPPER_SNAKE_CASE (`API_BASE_URL` , `DEFAULT_TIMEOUT`)

👨‍💻 Geliştirme Rehberi

Yeni Feature Ekleme

1. **Entity Oluştur**

```
// lib/domain/entities/my_entity.dart
@freezed
class MyEntity with _$MyEntity {
  const factory MyEntity({
    required String id,
    required String name,
  }) = _MyEntity;

  factory MyEntity.fromJson(Map<String, dynamic> json) =>
      _$MyEntityFromJson(json);
}
```

2. **DataSource Oluştur**

```
// lib/data/datasources/my_entity_remote_data_source.dart
@LazySingleton()
class MyEntityRemoteDataSource {
  MyEntityRemoteDataSource(this._dioClient);
  final DioClient _dioClient;

  // API methods
}
```

3. **Repository Oluştur**

```
// lib/data/repositories/my_entity_repository.dart
@LazySingleton()
class MyEntityRepository {
  MyEntityRepository(this._remoteDataSource);
  final MyEntityRemoteDataSource _remoteDataSource;
```

```
    // Business logic
  }
```

4. **Provider Oluştur**

```
// lib/presentation/providers/my_entity_provider.dart
@injectable
class MyEntityProvider extends BaseProvider {
  MyEntityProvider(this._repository);
  final MyEntityRepository _repository;

  // State management
}
```

5. **Screen Oluştur**

```
// lib/presentation/screens/my_entity_screen.dart
class MyEntityScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Consumer<MyEntityProvider>(
      builder: (context, provider, child) {
        // UI implementation
      },
    );
  }
}
```

Code Generation

Yeni entity'ler ekledikten sonra:

```
flutter packages pub run build_runner build --delete-conflicting-outputs
```

Best Practices

1. **Error Handling**: Her API çağrısında proper error handling
2. **Loading States**: UI'da loading ve error state'leri göster
3. **Caching**: Uygun yerlerde caching kullan
4. **Logging**: Development'ta logging, production'da kapat

5. **Testing**: Her feature için test yaz

🧪 Test Stratejisi

Test Yapısı

```
test/
├── unit/                    # Unit tests
│   ├── core/               # Core layer tests
│   └── utils/              # Utility tests
└── widget/                 # Widget tests
    └── widgets/            # Widget tests
```

Test Örnekleri

```dart
// Unit Test
void main() {
  group('AuthProvider Tests', () {
    late MockAuthRepository mockRepository;
    late AuthProvider provider;

    setUp(() {
      mockRepository = MockAuthRepository();
      provider = AuthProvider(mockRepository);
    });

    test('should login successfully', () async {
      // Arrange
      when(() => mockRepository.login(
        userNameOrEmail: any(named: 'userNameOrEmail'),
        password: any(named: 'password'),
      )).thenAnswer((_) async => Result.success(userData));

      // Act
      final result = await provider.login(
        userNameOrEmail: 'test@example.com',
        password: 'password',
      );

      // Assert
      expect(result, true);
```

```
      expect(provider.isAuthenticated, true);
    });
  });
}
```

Test Çalıştırma

```
# All tests
flutter test

# Specific test file
flutter test test/unit/core/auth_provider_test.dart
```

🚀 Build ve Deployment

Development Build

```
flutter run --debug
```

Release Build

```
# Android
flutter build apk --release

# iOS (on macOS)
flutter build ios --release
```

Environment-specific Builds

```
# Development
flutter build apk --dart-define=API_URL=http://dev.api.com

# Production
flutter build apk --dart-define=API_URL=https://api.eventup.com
```

🔧 Troubleshooting

Yaygın Problemler

1. **Code Generation Hatası**

```
flutter clean
flutter pub get
flutter packages pub run build_runner build --delete-conflicting-outp
```

2. **Dependency Injection Hatası**
   - `@injectable` annotation'ı kontrol et
   - Code generation'ı tekrar çalıştır
3. **Build Hatası**
   - Android SDK ve iOS certificates kontrol et
   - `flutter doctor` çalıştır

Debug Modu

```
// Debug logging enable
if (Environment.isDevelopment) {
  print('Debug info: $debugInfo');
}
```

📚 Ek Kaynaklar

- Flutter Documentation
- Clean Architecture
- Provider Package
- GetIt Package
- Injectable Package

👥 Katkıda Bulunma

1. Fork yapın
2. Feature branch oluşturun (`git checkout -b feature/amazing-feature`)
3. Commit yapın (`git commit -m 'Add amazing feature'`)
4. Push yapın (`git push origin feature/amazing-feature`)
5. Pull Request oluşturun

Commit Message Format

```
type(scope): description

feat(auth): add biometric authentication
fix(events): resolve event loading issue
docs(readme): update installation instructions
```

Bu dokümantasyon güncellenmelidir. Yeni feature'lar eklendikçe veya mimari değişiklikler yapıldıkça bu dokümantasyonu güncelleyin.